Aplicaciones de interfaz gráfica

UNICAM – CUR Matagalpa – Departamento de Ciencias, Tecnología y Salud

Fecha: octubre, 2026

Universidad Nacional Autónoma de Nicaragua, Managua · Centro Universitario Regional de Matagalpa – Departamento de Ciencias, Tecnología y Salud

Manual didáctico Encuentro N°1: Aplicaciones de interfaz gráfica

Autor: Heydy Judelki Mairena Palacios

Componente: Programación de aplicaciones de escritorio

Carrera: Ingeniería de Sistemas

Año: II año – II semestre

Octubre 2026

¡Universidad del Pueblo y para el Pueblo!

Elaborado por:	**Revisado por:**	**Autorizado por:**
Heydy Judelki Mairena Palacios	Erick Lanzas	Erick Lanzas

Presentación del material

Bienvenido y bienvenida al primer encuentro del componente Programación de Aplicaciones de Escritorio. Este material didáctico ha sido elaborado para acompañarle en el aprendizaje de las aplicaciones de interfaz gráfica en C# con Windows Forms, y da inicio a la construcción de AgroStock, la aplicación de escritorio que desarrollaremos durante el semestre para el inventario de un agroservicio.

A lo largo de estas páginas, comprenderá cómo está formada una aplicación de interfaz gráfica; cómo se colocan y configuran los controles de un formulario; cómo el programa responde a las acciones del usuario mediante eventos; y cómo se organizan los menús, las barras de herramientas y las cajas de diálogo. Al final del encuentro tendrá la versión 1 de AgroStock funcionando.

Objetivo de aprendizaje del encuentro

- Explicar la estructura de una aplicación de escritorio con interfaz gráfica en C# (Windows Forms): punto de entrada, formularios, controles y ciclo de eventos.
- Construir formularios con controles, manejo de eventos, menús, barras de herramientas y cajas de diálogo para resolver un problema de inventario de un agroservicio local.

Contenidos de este encuentro

- Unidad I. Aplicaciones de interfaz gráfica: estructura de una aplicación de interfaz gráfica; uso de controles en un formulario; manejo de eventos; menús y barras de herramientas; cajas de diálogo.

A. Entrada temática

Vivencia

Doña Martha atiende un agroservicio en la salida hacia San Ramón. Anota en un cuaderno cada quintal de fertilizante que entra y sale.

Cuando un productor pregunta si hay fórmula 18-46-0, ella revisa varias páginas; a fin de mes suma a mano y casi nunca coincide con lo que hay en bodega.

Antes de continuar, reflexione:

- ¿Qué información anota doña Martha de cada producto? Haga una lista de campos.
- Si tuviera que diseñar la pantalla para registrar un producto, ¿qué control usaría para cada campo: cuadro de texto, lista desplegable, número, fecha?
- ¿Qué debería pasar cuando ella pulsa «Guardar» y olvidó escribir el precio?
- ¿Qué programas de escritorio conoce en su comunidad (facturación, planillas, control de ventas)? ¿Qué tienen en común sus ventanas?

Responda las preguntas anteriores en su cuaderno y compártalas con su pareja.

Actividad SQA: en su cuaderno o en el móvil, elabore una tabla de tres columnas —Qué sé, Qué quiero saber, Qué aprendí— sobre las aplicaciones de interfaz gráfica. Complete únicamente las dos primeras columnas por ahora; la tercera la completará al cierre de este material.

Lo que esperamos aprender: al terminar este material podrá construir una aplicación de escritorio con ventanas, controles, eventos, menús y cajas de diálogo, y la aplicará al caso de doña Martha.

B. Desarrollo central

Fundamentación científica

1. Estructura de una aplicación de interfaz gráfica

Una aplicación de interfaz gráfica de usuario (GUI) presenta la información en ventanas y responde a lo que el usuario hace con el ratón y el teclado. A diferencia de un programa de consola, que ejecuta instrucciones de arriba hacia abajo, una aplicación GUI es dirigida por eventos: después de mostrar la ventana, espera a que ocurra algo (un clic, una tecla, un temporizador) y ejecuta el código asociado.

En .NET, la tecnología Windows Forms permite construir estas aplicaciones con el lenguaje C#. Un proyecto Windows Forms típico contiene:

Archivo	Función
AgroStock.csproj	Configuración del proyecto: versión de .NET (net8.0-windows), uso de Windows Forms y paquetes NuGet.
Program.cs	Punto de entrada. El método Main inicializa la aplicación y llama a Application.Run(new FrmPrincipal()).
FrmPrincipal.cs	Clase del formulario que hereda de Form. Aquí se escribe el código de los eventos.
FrmPrincipal.Designer.cs	Código generado por el diseñador visual: crea los controles y asigna sus propiedades en InitializeComponent().

```
internal static class Program
{
    [STAThread]
    private static void Main()
    {
        ApplicationConfiguration.Initialize(); // estilos visuales y DPI
        Application.Run(new Forms.FrmPrincipal()); // inicia el bucle de mensajes
    }
}
```

Application.Run inicia el bucle de mensajes: el sistema operativo coloca en una cola cada acción del usuario y la aplicación la convierte en un evento del control correspondiente. El atributo

[STAThread] indica que la interfaz se ejecuta en un único hilo (el hilo de la interfaz); lo retomaremos en la Unidad III.

Un formulario es una clase. Por eso aplicamos lo aprendido en Programación Orientada a Objetos: el formulario hereda de Form, sus controles son atributos privados y los manejadores de eventos son métodos.

Ejemplo aplicado — AgroStock: el formulario principal (FrmPrincipal) es una clase que hereda de Form; el punto de entrada solo lo muestra y deja que los eventos del usuario dirijan el programa.

2. Uso de controles en un formulario

Un control es un componente visual reutilizable. Cada control tiene propiedades (su estado: Text, Enabled, Visible), métodos (acciones: Focus(), Clear()) y eventos (avisos: Click, TextChanged). Elegir el control correcto evita errores de captura.

Control	Uso en AgroStock	Propiedad/evento clave
Label	Etiquetas de los campos	Text
TextBox	Código y nombre del producto	Text, MaxLength, PlaceholderText / TextChanged, KeyPress
ComboBox	Categoría (lista cerrada)	DropDownStyle = DropDownList, Items / SelectedIndexChanged
NumericUpDown	Precio y existencia	DecimalPlaces, Minimum, Maximum / ValueChanged
DateTimePicker	Fecha de vencimiento	Format, ShowCheckBox / ValueChanged
Button	Guardar, Cancelar	Text, DialogResult / Click
DataGridView	Lista de productos	DataSource, SelectionMode / CellDoubleClick

Para que la ventana se vea bien en cualquier tamaño de pantalla se usan las propiedades Dock (acoplar a un borde o llenar) y Anchor (mantener la distancia a los bordes), y contenedores como TableLayoutPanel (cuadrícula de filas y columnas) y FlowLayoutPanel (elementos uno tras otro).

```
var tabla = new TableLayoutPanel { Dock = DockStyle.Fill, ColumnCount = 2 };
tabla.ColumnStyles.Add(new ColumnStyle(SizeType.Absolute, 120)); // etiquetas
tabla.ColumnStyles.Add(new ColumnStyle(SizeType.Percent, 100));  // controles
tabla.Controls.Add(new Label { Text = "&Nombre:", AutoSize = true });
tabla.Controls.Add(txtNombre);
```

Convención de nombres: prefijo del tipo + propósito (txtNombre, cboCategoria, nudPrecio, btnGuardar, dgvProductos). Evite dejar button1 o textBox2: el código se vuelve ilegible.

Ejemplo aplicado — AgroStock: cada campo de la vivencia de doña Martha tiene su control: el nombre en un TextBox, la categoría en un ComboBox, el precio y la existencia en NumericUpDown y el vencimiento en un DateTimePicker.

3. Manejo de eventos

Un evento es un aviso que un objeto (el emisor) envía cuando algo ocurre. Otros objetos se suscriben con el operador += indicando el método que debe ejecutarse (el manejador). Todo manejador recibe dos parámetros: sender (quién generó el evento) y e (datos del evento, de tipo EventArgs o una clase derivada).

```
// Suscripción (en el constructor o en el diseñador)
btnGuardar.Click += BtnGuardar_Click;
txtCodigo.KeyPress += TxtCodigo_KeyPress;

// Manejador: solo letras, dígitos y guion; convierte a mayúscula
private void TxtCodigo_KeyPress(object? sender, KeyPressEventArgs e)
{
    if (char.IsControl(e.KeyChar)) return;          // permitir Retroceso
    if (!char.IsLetterOrDigit(e.KeyChar) && e.KeyChar != '-')
    {
        e.Handled = true;                            // se descarta la tecla
        return;
    }
    e.KeyChar = char.ToUpperInvariant(e.KeyChar);
}
```

Evento	¿Cuándo ocurre?	Ejemplo de uso
Load	Antes de mostrar el formulario por primera vez	Cargar datos iniciales
Click	Al pulsar un botón o elemento de menú	Guardar, eliminar
TextChanged	Cada vez que cambia el texto	Búsqueda mientras se escribe
KeyPress / KeyDown	Al pulsar una tecla	Filtrar caracteres, atajos (Ctrl+F)

Evento	¿Cuándo ocurre?	Ejemplo de uso
CellDoubleClick	Doble clic en una celda del DataGridView	Abrir el producto para editar
FormClosing	Antes de cerrar la ventana (se puede cancelar)	Confirmar salida
Tick (Timer)	Cada intervalo del temporizador	Reloj en la barra de estado

La validación de datos se hace en el evento del botón Guardar (o en Validating de cada control). El componente ErrorProvider muestra un icono junto al control con el mensaje de error, sin interrumpir con ventanas emergentes.

```
private void BtnGuardar_Click(object? sender, EventArgs e)
{
    errores.Clear();
    if (txtNombre.Text.Trim().Length < 3)
    {
        errores.SetError(txtNombre, "Escriba al menos 3 caracteres.");
        return;                          // el diálogo sigue abierto
    }
    Producto.Nombre = txtNombre.Text.Trim();
    DialogResult = DialogResult.OK;    // cierra el formulario modal
}
```

Ejemplo aplicado — AgroStock: si doña Martha olvida el nombre del producto, ErrorProvider se lo señala junto al cuadro de texto y la ventana permanece abierta para que lo corrija.

4. Menús y barras de herramientas

Los menús organizan todas las funciones de la aplicación; las barras de herramientas ofrecen acceso rápido a las más usadas; la barra de estado informa sin interrumpir. En Windows Forms se usan MenuStrip, ToolStrip y StatusStrip, cuyos elementos (ToolStripMenuItem, ToolStripButton, ToolStripStatusLabel) comparten el mismo modelo de eventos.

- El carácter & antes de una letra crea la tecla de acceso: "&Archivo" se abre con Alt+A.
- ShortcutKeys asigna un atajo global: Keys.Control | Keys.N para Nuevo.
- ToolTipText muestra una ayuda al pasar el ratón sobre un botón.
- ContextMenuStrip crea el menú del clic derecho y se asigna a la propiedad ContextMenuStrip del control.

```
var archivo = new ToolStripMenuItem("&Archivo");
archivo.DropDownItems.Add(new ToolStripMenuItem("&Nuevo producto", null, (_, _) => Nuevo(),
Keys.Control | Keys.N));
archivo.DropDownItems.Add(new ToolStripSeparator());
archivo.DropDownItems.Add(new ToolStripMenuItem("&Salir", null, (_, _) => Close()));
menu.Items.Add(archivo);

var barra = new ToolStrip();
barra.Items.Add(new ToolStripButton("Nuevo", null, (_, _) => Nuevo()) { ToolTipText = "Nuevo
(Ctrl+N)" });
```

Principio de diseño: toda acción de la barra de herramientas debe existir también en el menú. Así el usuario que no usa ratón puede llegar a ella con el teclado (accesibilidad).

5. Cajas de diálogo

Una caja de diálogo es una ventana secundaria que solicita información o confirma una acción. Es modal cuando bloquea la ventana principal hasta que se cierra (ShowDialog) y no modal cuando permite seguir trabajando (Show).

Diálogo	Para qué	Resultado
MessageBox.Show	Informar, advertir, confirmar	DialogResult.Yes / No / OK / Cancel
OpenFileDialog	Elegir un archivo para abrir	FileName
SaveFileDialog	Elegir dónde guardar	FileName
FontDialog / ColorDialog	Elegir fuente o color	Font / Color
Formulario propio con ShowDialog	Capturar datos (FrmProducto)	DialogResult + propiedades del formulario

```
var p = new Producto();
using var dlg = new FrmProducto(p);          // using libera la ventana al terminar
if (dlg.ShowDialog(this) == DialogResult.OK)  // modal: espera la respuesta
{
    productos.Add(p);
}

var r = MessageBox.Show(this, $"¿Eliminar «{p.Nombre}»?", "Confirmar",
    MessageBoxButtons.YesNo, MessageBoxIcon.Question, MessageBoxDefaultButton.Button2);
if (r == DialogResult.Yes) productos.Remove(p);
```

En el formulario propio, AcceptButton hace que Enter equivalga a Guardar y CancelButton que Esc equivalga a Cancelar. El botón Cancelar con DialogResult = Cancel cierra el diálogo automáticamente.

6. Integración: AgroStock v1

El código completo de AgroStock v1 está en la carpeta de recursos del Encuentro 1 (sitio del curso). Estructura del proyecto:

```
AgroStock.E01/
├── AgroStock.E01.csproj
├── Program.cs              ← punto de entrada
├── Models/Producto.cs      ← clase del dominio (POO)
└── Forms/
    ├── FrmPrincipal.cs     ← menú, barras, DataGridView, búsqueda, exportar CSV
    └── FrmProducto.cs      ← diálogo modal con validación
```

Los datos se guardan en una BindingList<Producto>: una lista que avisa a la cuadrícula cada vez que se agrega, cambia o elimina un elemento. En el Encuentro 2 la reemplazaremos por una base de datos SQLite sin cambiar la interfaz.

Ejemplo aplicado — AgroStock: el cuaderno de doña Martha vs. AgroStock v1

Aspecto	Cuaderno	AgroStock v1
Búsqueda	Revisar varias páginas	Búsqueda sobre la cuadrícula de productos
Captura de datos	Texto libre, sin controles	Controles adecuados (ComboBox, NumericUpDown, DateTimePicker)
Datos incompletos	Nadie avisa que falta el precio	Validación con ErrorProvider en el formulario modal
Fin de mes	Suma a mano	Exportación de la lista a CSV
Almacenamiento	Páginas dispersas	Lista BindingList enlazada a la cuadrícula

Ejercitación

Las siguientes actividades fomentan el aprendizaje autónomo e investigativo y se realizan de forma individual y colaborativa.

Actividad individual

- Complete la tercera columna de su tabla SQA («Qué aprendí») con las ideas clave de la fundamentación científica.
- Resuelva la autoevaluación sin mirar el material y luego verifique.

1. ¿Qué hace Application.Run(new FrmPrincipal())?

a) Compila el proyecto

b) Muestra el formulario e inicia el bucle de mensajes

c) Crea la base de datos

d) Cierra la aplicación

2. ¿Qué control es el más adecuado para capturar la categoría de un producto de una lista cerrada?

a) TextBox

b) Label

c) ComboBox con DropDownList

d) Button

3. En un manejador KeyPress, ¿qué logra e.Handled = true?

a) Cierra el formulario

b) Descarta la tecla pulsada

c) Convierte la tecla a mayúscula

d) Guarda el producto

4. ¿Qué operador suscribe un método a un evento en C#?

a) =

b) ==

c) +=

d) =>

5. ¿Cuál es la diferencia entre ShowDialog() y Show()?

a) No hay diferencia

b) ShowDialog abre una ventana modal y devuelve DialogResult

c) Show es más rápido

d) ShowDialog solo sirve para MessageBox

6. ¿Qué significa el & en el texto "&Archivo" de un menú?

a) Es un error

b) Crea la tecla de acceso Alt+A

c) Pone la palabra en negrita

d) Agrega un separador

7. ¿En qué evento se puede cancelar el cierre de una ventana?

a) Load

b) Click

c) FormClosing

d) TextChanged

Actividad colaborativa (en grupos)

El trabajo interencuentro se retoma en la BOA 1. Debe realizarse en grupo y entregarse en el encuentro 2

1. Completar AgroStock v1 según la lista de cotejo (menú, barras, formulario modal, validación, exportación CSV).
2. Investigación de campo: visitar o entrevistar un negocio local e identificar 3 procesos que AgroStock podría mejorar (una página, con evidencia).

Práctica de programación

Resuelva de forma individual los siguientes ejercicios. Cree un proyecto Windows Forms por ejercicio.

Nivel 1 – Básico

1. Cree un proyecto Windows Forms «Saludo» con un TextBox, un Button y un Label. Al pulsar el botón, el Label muestre «Bienvenido, <nombre>». Si el nombre está vacío, muestre un MessageBox de advertencia.

2. Contador de existencias: dos botones (+ y −) modifican un número en un Label. El botón − se deshabilita cuando el valor llega a cero (propiedad Enabled).

Nivel 2 – Intermedio

1. Calculadora de fertilizante: el usuario ingrese el área en manzanas (NumericUpDown) y la dosis en quintales por manzana; al cambiar cualquier valor (ValueChanged) se muestre el total de quintales y su costo usando un precio fijo.
2. Agregue a su calculadora un MenuStrip con Archivo → Salir (con confirmación en FormClosing) y Ayuda → Acerca de.

Nivel 3 – Avanzado (opcional)

1. En AgroStock v1, agregue un ContextMenuStrip a la cuadrícula con Editar y Eliminar.
2. Coloree en naranja las filas con existencia menor que 10 usando el evento CellFormatting del DataGridView.

Entrega

Envíe el código de los ejercicios (tarea «Ejercitación E1») y el proyecto AgroStock v1 sin las carpetas bin y obj, Junto con AgroStock v1, entregue la investigación de campo: visitar o entrevistar un negocio local e identificar 3 procesos que AgroStock podría mejorar (una página, con evidencia).

Lista de verificación: marque lo que ya logró.

☐ Instalé Visual Studio con «Desarrollo de escritorio de .NET»

☐ Creé y ejecuté un proyecto Windows Forms

☐ Diseñé el formulario de producto con controles adecuados

☐ Programé al menos 5 eventos distintos

☐ Agregué MenuStrip, ToolStrip y StatusStrip

☐ Usé MessageBox, SaveFileDialog y un formulario modal propio

☐ Subí la Ejercitación E1

☐ Entregué AgroStock v1 y el informe de campo

C. Cierre y consolidación del aprendizaje

Complementación

El dominio de las aplicaciones de interfaz gráfica va más allá de memorizar nombres de controles o de eventos. Reflexione sobre las siguientes preguntas, que conectan lo aprendido con su realidad como futuro(a) ingeniero(a) en sistemas:

Preguntas para reflexionar

- **Conexión con la práctica:** Piense en los negocios de su comunidad —un agroservicio, una pulpería, una cooperativa—. ¿De qué manera una aplicación de escritorio con formularios, validación y menús puede sustituir el cuaderno y reducir los errores de registro?
- **Transferencia del aprendizaje:** Ya conoce cómo funciona una aplicación dirigida por eventos. ¿Cómo podría aplicar los controles, los eventos y los diálogos al diseño de otras aplicaciones, como planillas, facturación o control de ventas?
- **Relación con el eje Madre Tierra:** ¿Cómo puede una aplicación sencilla reducir el uso de papel y los viajes innecesarios de los productores? Escriba tres ideas.

Sitios web para ampliar sus conocimientos

Le comparto una selección de recursos en línea, de fuentes confiables, para que pueda seguir perfeccionando sus habilidades:

Recurso	Descripción	Enlace
Microsoft Learn – Tutorial: Creación de una aplicación de Windows Forms con .NET	Tutorial paso a paso para crear una primera aplicación de Windows Forms con .NET en Visual Studio.	https://learn.microsoft.com/es-es/dotnet/desktop/winforms/get-started/create-app-visual-studio
Microsoft Learn – Información general sobre eventos (Windows Forms .NET)	Explica cómo funcionan los eventos y los manejadores en Windows Forms, base de todo lo trabajado en este encuentro.	https://learn.microsoft.com/es-es/dotnet/desktop/winforms/forms/events

Microsoft Learn – Guía de escritorio (Windows Forms .NET)	Documentación general de Windows Forms: formularios, controles, menús y cuadros de diálogo, para profundizar cada tema.	https://learn.microsoft.com/es-es/dotnet/desktop/winforms/

Glosario del encuentro

Término	Definición
GUI	Interfaz gráfica de usuario: ventanas, botones y menús en lugar de comandos de texto.
Control	Componente visual reutilizable (botón, cuadro de texto) con propiedades, métodos y eventos.
Evento	Aviso que emite un objeto cuando algo ocurre; otros objetos se suscriben para reaccionar.
Manejador de eventos	Método que se ejecuta cuando ocurre un evento; recibe sender y EventArgs.
Formulario modal	Ventana que bloquea a su ventana dueña hasta cerrarse (ShowDialog).
DialogResult	Valor que indica cómo se cerró un diálogo (OK, Cancel, Yes, No).
Dock / Anchor	Propiedades que controlan cómo un control se acomoda al cambiar el tamaño de la ventana.
BindingList	Lista que notifica sus cambios a los controles enlazados (DataGridView).

Cierre y transferencia del aprendizaje

Construir una interfaz no es solo colocar controles en una ventana: es pensar en la persona que la usará. Un formulario con el control adecuado, una validación oportuna y un menú accesible pueden convertir el cuaderno de doña Martha en una herramienta confiable, y esa es la diferencia entre una pantalla que estorba y una que ayuda.

Así como doña Martha podrá pasar de las páginas del cuaderno a una pantalla ordenada, usted inicia hoy su propio camino en la programación de aplicaciones de escritorio, y AgroStock v1 es el primer paso de una aplicación que seguiremos construyendo en los próximos encuentros.

Conclusiones clave del encuentro

1. Una aplicación GUI es dirigida por eventos: muestra la ventana y reacciona a las acciones del usuario.
2. Cada formulario es una clase; cada control es un objeto con propiedades, métodos y eventos.
3. Elegir el control adecuado (ComboBox, NumericUpDown, DateTimePicker) previene errores de captura.
4. Menús, barras y atajos hacen la aplicación rápida y accesible; los diálogos confirman y capturan datos.
5. AgroStock v1 integra lo aprendido: formulario principal con menú, barras y cuadrícula, formulario modal con validación y datos en una BindingList que luego, en el Encuentro 2, será reemplazada por una base de datos SQLite sin cambiar la interfaz.

Transferencia del aprendizaje

Los conocimientos que ha adquirido son un pilar que trasciende este componente curricular. Los principios de interfaz gráfica, eventos y diálogos son directamente transferibles a su práctica profesional:

1. Un formulario es una clase: lo aprendido en Programación Orientada a Objetos se aplica directamente al construir ventanas, controles y manejadores de eventos.

2. El modelo de eventos (un emisor que avisa, un manejador suscrito que responde) se repite en otras tecnologías de interfaz, por lo que comprenderlo hoy facilita aprender otras después.
3. Los programas de escritorio que existen en su comunidad —facturación, planillas, control de ventas— usan las mismas ventanas, menús y cuadros de diálogo que usted ya sabe construir.

“La práctica constante es la clave”. Aplíquese a los ejercicios del encuentro y a AgroStock, y estará construyendo una competencia que le acompañará durante toda su carrera profesional.

Referencias

Ceballos Sierra, F. J. *Enciclopedia de Microsoft Visual C#*. Editorial RA-MA.

ClosedXML (2024). *Documentación*. https://docs.closedxml.io/

Deitel, H. M. y Deitel, P. J. *C# Cómo programar*. Editorial Pearson, Prentice Hall.

Microsoft (2024). *Entity Framework Core*. Microsoft Learn. https://learn.microsoft.com/es-es/ef/core/

Microsoft (2024). *Guía de escritorio (Windows Forms .NET)*. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/desktop/winforms/

Microsoft (2024). *Información general sobre eventos (Windows Forms .NET)*. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/desktop/winforms/forms/events

Microsoft (2024). *Microsoft.Data.Sqlite: información general*. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/standard/data/sqlite/

Microsoft (2024). *Realización de solicitudes HTTP con la clase HttpClient*. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/fundamentals/networking/http/httpclient

Microsoft (2024). *Subprocesamiento administrado y programación asincrónica*. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/standard/threading/

Microsoft (2024). *Tutorial: Creación de una aplicación de Windows Forms con .NET*. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/desktop/winforms/get-started/create-app-visual-studio

QuestPDF (2024). *Documentación*. https://www.questpdf.com/

SQLite Consortium (2024). *SQL As Understood By SQLite*. https://www.sqlite.org/lang.html



¡Universidad
del Pueblo y para el Pueblo!

www. unan.edu.ni